

How a rupee moves through the simulator

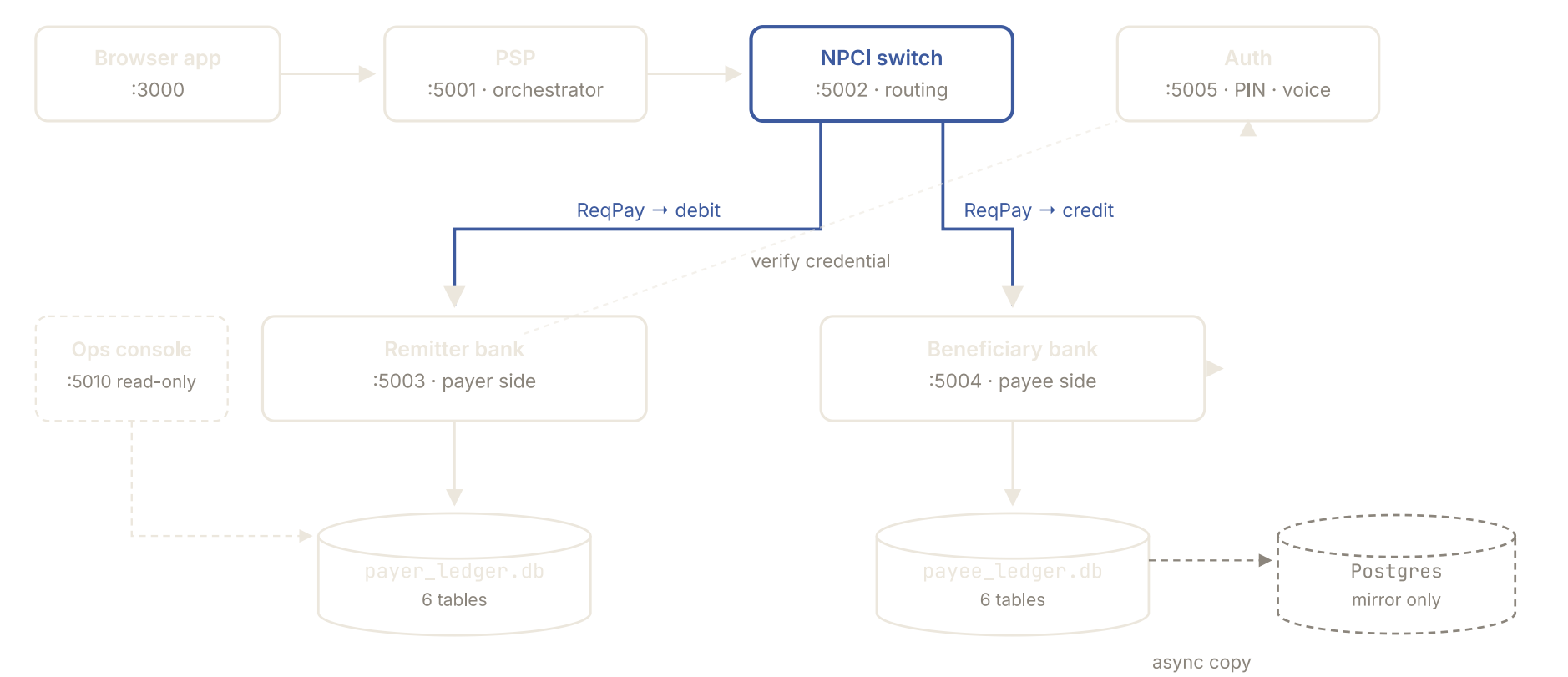
Every hop, every database, every table and every ledger entry — for a payment, a refund and a settlement cycle. Drawn from the running code, not from the design doc.

6 services · 2 bank ledgers (SQLite) · 1 reporting mirror (Postgres)
· amounts stored as integer paise

MAP

The pieces, and who owns which database

Each bank keeps its own books in its own file. No service reads another's database — they talk over HTTP only. That isolation is the whole point: it is what makes a half-finished payment possible, and therefore what the ledger has to survive.



Solid lines are request paths. The switch is the only component that talks to both banks — they never talk to each other. The Postgres mirror is written to asynchronously and nothing reads back from it to make a decision.

WHAT LIVES IN EACH BANK'S DATABASE

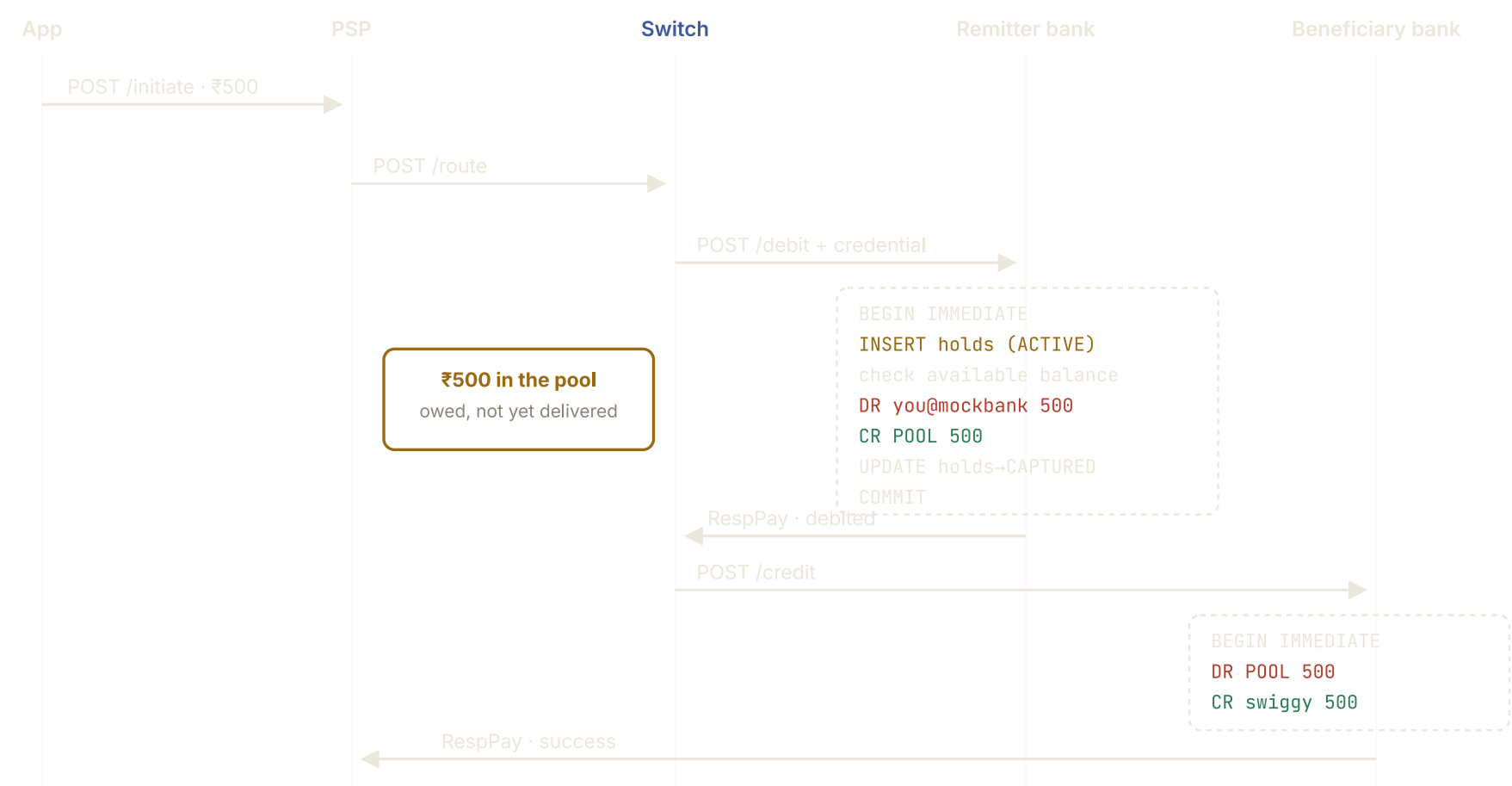
TABLE	HOLDS	WRITTEN WHEN
ledger_accounts	Chart of accounts — every customer, plus the bank's own P00L , N0STRO , SUSPENSE , OPENING	Account opened
journal_entries	One row per money event: event_type , txn_ref , and reverses pointing at a prior entry	Every debit, credit, reversal, settlement
postings	The actual money. Two or more rows per entry: direction DR/CR and amount_paise	Same moment as the entry, same transaction
holds	Earmarked funds not yet posted — the gap between ledger and available balance	Debit begins; closed when it posts
accounts	Identity, plus a <i>cached</i> balance column	Refreshed after each posting — never the source of truth
transactions	Payment-level state: status , settlement_batch_id , failure_reason	Each stage transition

Balances are never stored. A balance is SUM(CR) - SUM(DR) over postings, computed on every read. The accounts.balance column is a cache kept in step for convenience; if it ever disagrees with the journal, the journal is right.

FLOW 1

A payment: ₹500 from you@mockbank to swiggy@mockbank2

Four balanced journal entries across two databases. Watch where the money sits between step 4 and step 6 — it is in neither person's account.



Between the debit and the credit the money belongs to neither party: it sits in the remitter bank's pool account as an inter-bank obligation. That is exactly why a failed credit leg is recoverable.

STEP BY STEP, WITH THE TABLES TOUCHED

#	CALL	DATABASE → TABLE	WHAT IS WRITTEN
1	POST :5001/initiate	PSP memory	Generates <code>txn_id</code> , checks the daily velocity limit, status <code>initiated</code>
2	POST :5002/route	switch memory → <code>route_logs</code>	Resolves the <code>@handle</code> suffix to each bank's URL and logs the hop
3	POST :5005/step-up/verify	<code>voice_embeddings</code>	Verifies PIN, passkey or voice. Happens <i>before</i> any lock is taken
4	POST :5003/debit one transaction	<code>payer_ledger.holds</code>	INSERT earmark ₹500, status ACTIVE
		<code>payer_ledger.postings</code>	read SUM → available balance = ledger – holds
		<code>payer_ledger.journal_entries</code>	INSERT DEBIT_LEG
		<code>payer_ledger.postings</code>	DR MOCKBANK:you@mockbank 50000 paise CR MOCKBANK:POOL 50000 paise
		<code>payer_ledger.transactions</code>	INSERT status debited ; holds → CAPTURED; cache refreshed
5	POST :5004/credit one transaction	<code>payee_ledger.journal_entries</code>	INSERT CREDIT_LEG
		<code>payee_ledger.postings</code>	DR MOCKBANK2:POOL 50000 paise CR MOCKBANK2:swiggy@mockbank2 50000 paise
		<code>payee_ledger.transactions</code>	INSERT status credited ; cache refreshed
6	GET :5001/txn/{id}	PSP memory	App polls until the status is terminal

THE BALANCES AFTER EACH LEG

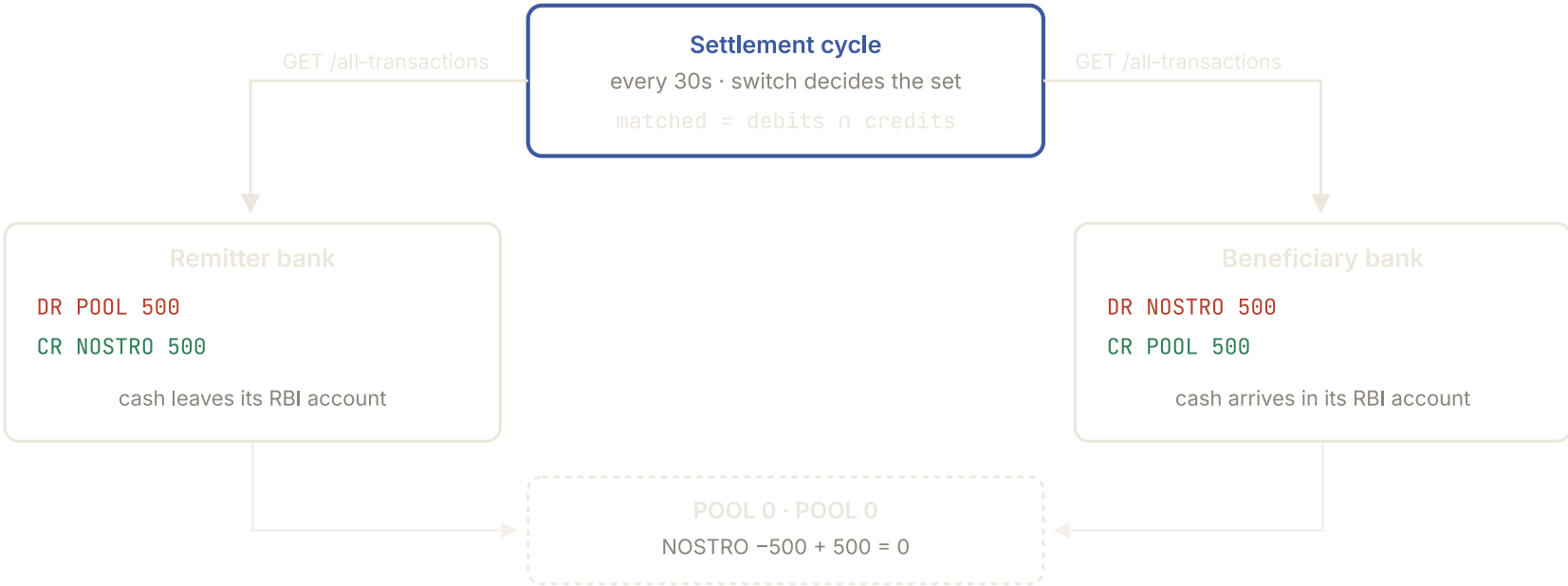
Remitter bank books			Beneficiary bank books		
you@mockbank	-500		P00L	-500	
P00L	+500		swiggy@...	+500	
net	0	✓	net	0	✓

Each bank's books balance on their own, at every instant. The imbalance lives *between* them: the remitter's pool is **+500** and the beneficiary's is **-500**. Those two numbers must always sum to zero, and the ops console shows that sum as its headline figure.

FLOW 2

Settlement: the pools empty into the RBI accounts

Every 30 seconds the switch asks both banks for their completed payments, matches them by transaction id, and tells each bank to clear its position. Until this runs, the pools only grow.



Only payments whose *both* legs completed enter the cycle. A debit with no matching credit stays in the pool, where a reversal can still reach it.

CALL	DATABASE → TABLE	WHAT IS WRITTEN
POST :5003/settlement/post	payer_ledger.journal_entries + postings	SETTLEMENT entry · DR POOL / CR NOSTRO
	payer_ledger.transactions	UPDATE status → success , stamp settlement_batch_id
POST :5004/settlement/post	payee_ledger.journal_entries + postings	SETTLEMENT entry · DR NOSTRO / CR POOL
	payee_ledger.transactions	UPDATE status → success , stamp settlement_batch_id
— mirror —	Postgres settlement_batches	Cycle summary for reporting

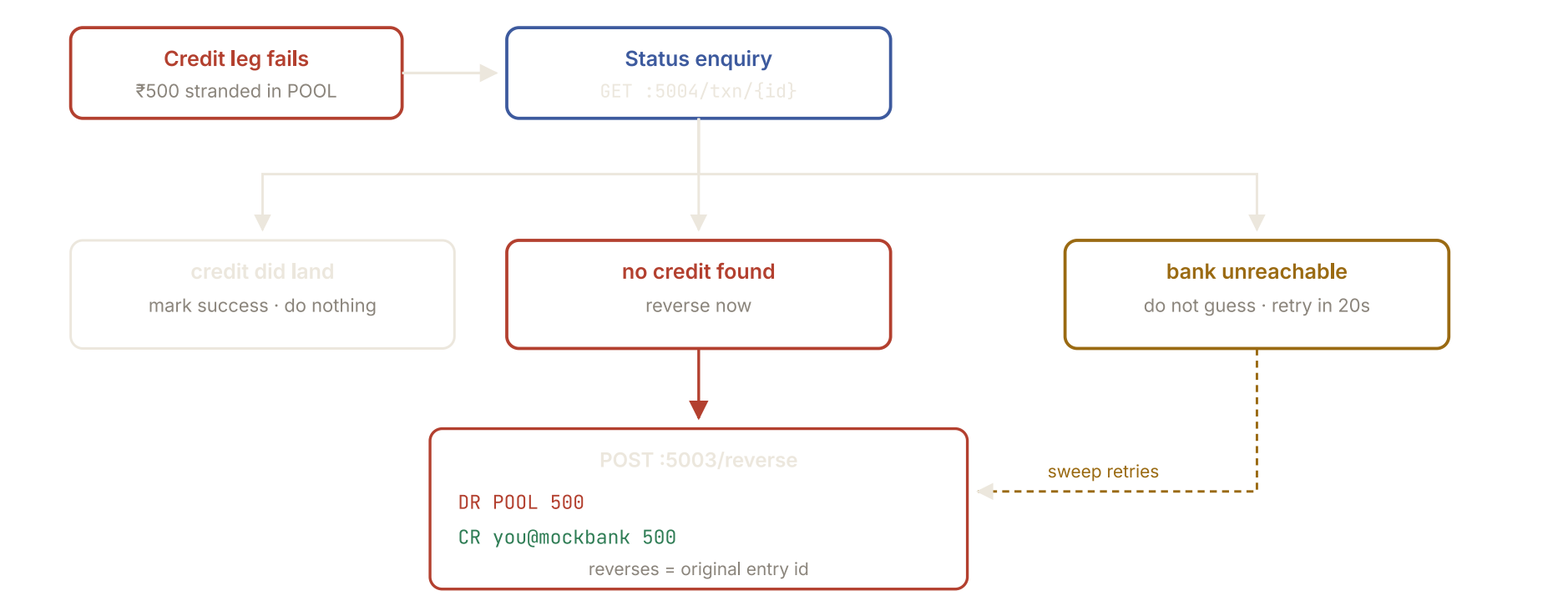
Why the matching matters. An earlier version settled whatever sat in the pool. A debit whose credit never arrived got swept into NOSTRO, past the point a reversal could reach it — the money-destruction bug one layer down. Settling only matched pairs is what prevents that.

FLOW 3

A refund: the credit leg fails

The payer has been debited and the beneficiary bank rejects the credit. The money is in

the remitter's pool. Nothing is deleted — the refund is a new entry pointing back at the original.



A timeout is not a failure. Reversing a credit that actually succeeded would create money, so the system refuses to reverse until it has confirmed the credit is absent — and retries every 20 seconds until it can.

CALL	DATABASE → TABLE	WHAT IS WRITTEN
GET :5004/txn/{id}	payee_ledger.transactions	Read only — did the credit land?
POST :5003/reverse	payer_ledger.journal_entries	Reads the original DEBIT_LEG , refuses if already reversed
	payer_ledger.journal_entries	INSERT REVERSAL with reverses = original entry_id
	payer_ledger.postings	DR MOCKBANK:POOL · CR MOCKBANK:you@mockbank
	payer_ledger.transactions	UPDATE status → reversed

WHAT THE CUSTOMER'S STATEMENT SHOWS

DEBIT_LEG	DR 500.00	45,450.00	UPI debit → swiggy@mockbank2
REVERSAL	CR 500.00	45,950.00	Reversal — credit leg failed

Both lines stay on the statement, exactly as a real bank statement would show them. The original entry is never edited or deleted — that is what append-only means, and it is why the two figures can always be audited against each other.

RULES

The four invariants that hold all of it together

- 1 Every entry balances. SUM(DR) = SUM(CR) is checked on every single write, not by a nightly job. An entry that does not balance is rejected, so the system cannot invent or lose money.
- 2 Each bank's books balance alone. No journal entry ever spans two databases. The inter-bank imbalance lives in the two pool accounts, never inside an entry.
- 3 The pools must cancel. remitter POOL + beneficiary POOL = 0. Any other number means value is in flight — or unreconciled, which is the same thing left too long.
- 4 NOSTRO nets to zero. After settlement, what one bank paid out of its RBI account is exactly what the other received.

Amounts are stored as integer paise throughout. 0.1 + 0.2 is 30 paise, not 0.30000000000000004.

